Politechnika Śląska

Wydział Górnictwa, Inżynierii Bezpieczeństwa
i Automatyki Przemysłowej



Prototyp projektu:
Optymalizacja energetyczna sterowania napędem elektrycznym z wykorzystaniem programowania dynamicznego

Autor: Dawid Saletnik
Kierunek: Automatyka i Informatyka Przemysłowa
Specjalizacja: Cyberbezpieczeństwo

Spis treści

1. Wprowadzenie 3
   a) Cel pracy 3
   b) Zakres i metodyka pracy 3
   c) Wkład własny 3
   d) Narzędzia 4
2. Zagadnienia teoretyczne 5
   a) Model matematyczny napędu 5
   b) Funkcja celu 5
   c) Optymalizacja dynamiczna 6
3. Projekt - wersja bez modelu Simulink. 7
   a) Skrypt główny – main.m 7
   b) Skrypt określający parametry projektu – params.m 7
   c) Skrypt symulujący napęd w postaci ciągłej przed optymalizacją – simulate_baseline.m 8
   d) Skrypt symulujący model napędu w postaci dyskretnej – model_discrete.m 8
   e) Skrypt rozwiązujący problem optymalizacji dynamicznej – dp_solver.m 8
   f) Skrypt symulujący napęd zoptymalizowany w postaci dyskretnej – simulate_dp.m 8
   g) Skrypt wyświetlający wyniki symulacji w postaci charakterystyk – plots.m 9
   h) Wyniki symulacji 9
4. Projekt – wersja z modelem Simulink 11
   a) Skrypt główny – main.m 11
   b) Skrypt określający parametry projektu – params.m 11
   c) Model Simulink symulujący napęd przed optymalizacją oraz po optymalizacji – system_model.slx 12
   d) Skrypt obsługujący model Simulink – run_simulink.m 13
   e) Skrypt rozwiązujący problem optymalizacji dynamicznej – dp_solver.m 14
   f) Skrypt wyświetlający wyniki symulacji w postaci charakterystyk – plots.m 15
   g) Wyniki symulacji 16

1. Wprowadzenie

a) Cel pracy

Główny cel pracy:

Celem proponowanego tematu pracy jest sformułowanie i rozwiązanie problemu optymalizacji energetycznej sterowania napędem elektrycznym wykorzystując w tym celu programowanie dynamiczne oraz analizę kompromisów pomiędzy zużyciem energii a jakością realizacji zadania sterowania.

Cele szczegółowe:

- Opracowanie modelu dynamicznego napędu
- Sformułowanie funkcji celu minimalizującej zużycie energii
- Implementacja algorytmu programowania dynamicznego w środowisku MATLAB
- Walidacja rozwiązania w środowisku Simulink
- Porównanie z klasycznymi metodami sterowania
- Analiza wpływu parametrów algorytmu na wynik optymalizacji

b) Zakres i metodyka pracy

Model obiektu:

- Uproszczony model napędu elektrycznego
- Uwzględnienie strat mechanicznych

Problem optymalizacji:

- Zmienne decyzyjne: moment napędowy
- Funkcja celu: energia + koszt końcowy
- Ograniczenia: dynamika, saturacje

Metody rozwiązania:

- Programowanie dynamiczne (DP)
- Dyskretyzacja czasu i przestrzeni stanu

Walidacja:

- Symulacje MATLAB
- Implementacja modelu w Simulinku
- Porównanie wyników

c) Wkład własny

Wkład własny polega na:

- Samodzielnym sformułowaniu problemu optymalizacji energetycznej sterowania napędem
- Doborze i analizie funkcji celu oraz jej parametrów
- Implementacji algorytmu programowania dynamicznego
- Przeprowadzeniu analizy porównawczej oraz interpretacji uzyskanych wyników

d) Narzędzia

Wykorzystane narzędzia:

- MATLAB
- SIMULINK

2. Zagadnienia teoretyczne

a) Model matematyczny napędu

Równanie dynamiki dla napędu o stałym wzbudzeniu:

$$\mathrm{J}\frac{\mathrm{d}\omega}{\mathrm{dt}} = \mathrm{M_n} - \mathrm{M_{obc}} - \mathrm{B} * \omega$$

Gdzie:

- J – moment bezwładności
- $\mathrm{M_n}$ – moment napędowy (współczynnik decyzyjny)
- $\mathrm{M_{obc}}$ – moment obciążenia zewnętrznego (dla wersji wstępnej projektu przyjęto $\mathrm{M_{obc}} = 0$)
- B – współczynnik tarcia (straty mechaniczne), $\mathrm{B} * \omega$ to moment tarcia

Postać zdyskretyzowana metodą Eulera:

$$\omega_{\mathrm{i+1}} = \omega_{\mathrm{i}} + \frac{\Delta \mathrm{t}}{\mathrm{J}}(\mathrm{M_{n,i}} - \mathrm{M_{obc,i}} - \mathrm{B} * \omega_{\mathrm{i}})$$

Gdzie:

- $\omega_{\mathrm{i+1}}$ – częstość kołowa układu w kroku i+1
- ω_{i} – częstość kołowa układu w kroku i+1
- J – moment bezwładności
- $\mathrm{M_{n,i}}$ – moment napędowy w roku i (współczynnik decyzyjny)
- $\mathrm{M_{obc,i}}$– moment obciążenia zewnętrznego w kroku i (dla wersji wstępnej projektu przyjęto $\mathrm{M_{obc,i}} = 0$)
- B – współczynnik tarcia (straty mechaniczne), $\mathrm{B} * \omega_i$ to moment tarcia w kroku i

b) Funkcja celu

Funkcja kosztu kroku k w dyskretnym programowaniu dynamicznym

$$\mathrm{J_k} = \sum_{\mathrm{i=k}}^{\mathrm{N-1}} \mathrm{L}(\mathrm{x_i}, \mathrm{u_i}) + \Phi(\mathrm{x_N})$$

$$\mathrm{L}(\mathrm{x_i}, \mathrm{u_i}) = \underbrace{M_{n,i} * \omega_i * \Delta t}_{\text{Energia}} + \underbrace{\beta * (\omega_i - \omega_{doc})^2}_{\text{Bieżący błąd}}$$

Gdzie:

- $\mathrm{J_k}$ – koszt kroku k
- L – zużycie energii (koszt kroku)
- Φ – kara za nieosiągnięcie zadanej prędkości lub pozycji na koniec symulacji (koszt końcowy)
- $\mathrm{x_i}$ – zmienna stanu w kroku i
- $\mathrm{u_i}$ – zmienna decyzyjna (sterowanie) w kroku i
- x_N – zmienna stanu w kroku i

c) Optymalizacja dynamiczna

Równanie Bellmana dla każdego kroku wstecz:

$$V(x_k, k) = \min_{u_k \epsilon U}\{L(x_k, u_k) + V(x_{k+1}, k+1)\}$$

Gdzie:

- $V(x_k, k)$– minimalny skumulowany koszt od kroku k do końca symulacji
- $L(x_k, u_k)$ – koszt przejścia (zużycie energii), zdefiniowany jako $M_{n,i} * \omega_i * \Delta t$
- $V(x_{k+1}, k+1)$– optymalny koszt następnego kroku, który został już obliczony (zasada indukcji wstecznej)
- x_{k+1} – zmienna stanu w kroku k+1
- x_k – zmienna stanu w kroku k
- u_k – zmienna decyzyjna (sterowanie) w kroku k

3. Projekt - wersja bez modelu Simulink.

a) Skrypt główny – main.m

```
clear;clc;

p = params();
Uopt = dp_solver(p);
[omega_dp, T_dp] = simulate_dp(p, Uopt);
[omega_base, T_base] = simulate_baseline(p);
plots(p, omega_dp, T_dp, omega_base, T_base);
```

b) Skrypt określający parametry projektu – params.m

```
function p = params()
  p.J  = 0.01;
  p.B  = 0.1;
  p.dt = 0.05;
  p.omega_0 = 0;
  p.T_base_val = 0;
  p.beta = 5;
  p.Tmax = 2;
  p.omega_max = 12;
  p.omega_target = 10;
  p.N = 40;
  p.omega_grid = linspace(0, p.omega_max, 60);
  p.T_grid     = linspace(0, p.Tmax, 30);
  p.alpha_terminal = 100;
end
```

Opis parametrów:

p.J - moment bezwładności wału napędowego – określa opór układu przy próbie zmiany prędkości

p.B – współczynnik tarcia lepkościowego – reprezentuje straty mechaniczne

p.dt – krok czasu – określa częstotliwość podejmowania przez algorytm decyzji o zmianie momentu napędowego

p,omega_0 – warunek początkowy – określa wartość prędkości na początku symulacji

p.T_base_val – stała moment napędowy – określa stałą wartość momentu napędowego

p.beta – waga kary – określa wagę kary jaką ponosi układ za błąd trakcie ruchu

p.Tmax – saturacja – określa maksymalny dopuszczalny moment napędowy

p.omega_max – zakres prędkości – określa zakres prędkości wykorzystywany przy budowie siatki stanów

p.omega_target – prędkość zadana – określa punkt, który napęd powinien osiągnąć na końcu procesu

p.N – horyzont czasowy – określa liczbę kroków wykonywanych podczas symulacji. Całkowity czas symulacji to p.N * p.dt

p.omega_grid – siatka prędkości – określa zbiór dopuszczalnych stanów, które może osiągnąć napęd podczas symulacji

p.T_grid – siatka momentów – określa zbiór dopuszczalnych decyzji sterujących, które algorytm może wybrać w trakcie symulacji

p.alpha_terminal – waga kosztu końcowego – decyduje czy ważniejsze jest osiągnięcie idealnej wartości prędkości docelowej (wysoka wartość), czy oszczędność energii (niska wartość)

c) **Skrypt symulujący napęd w postaci ciągłej przed optymalizacją – simulate_baseline.m**

```
function [omega_traj, T_traj] = simulate_baseline(p)
    omega_traj = zeros(1, p.N + 1);
    T_traj = zeros(1, p.N);
    omega_traj(1) = p.omega_0;
    for k = 1:p.N
        T_traj(k) = p.T_base_val;
        omega_traj(k+1) = model_discrete(omega_traj(k),T_traj(k),p);
    end
end
```

d) **Skrypt symulujący model napędu w postaci dyskretnej – model_discrete.m**

```
function omega_next = model_discrete(omega, T, p)
    omega_next = omega + p.dt*(1/p.J)*(T - p.B*omega);
end
```

e) **Skrypt rozwiązujący problem optymalizacji dynamicznej – dp_solver.m**

```
function Uopt = dp_solver(p)
    V = inf(length(p.omega_grid), p.N+1);
    Uopt = zeros(length(p.omega_grid), p.N);

    V(:, end) = p.alpha_terminal * ...
                (p.omega_grid - p.omega_target).^2;

    for k = p.N:-1:1
        for i = 1:length(p.omega_grid)
            omega = p.omega_grid(i);
            for u = 1:length(p.T_grid)
                T = p.T_grid(u);
                omega_next = model_discrete(omega, T, p);
                [~, j] = min(abs(p.omega_grid - omega_next));
                cost = T*omega*p.dt + V(j, k+1);
                if cost < V(i, k)
                    V(i, k) = cost;
                    Uopt(i, k) = T;
                end
            end
        end
    end
endf
```

f) **Skrypt symulujący napęd zoptymalizowany w postaci dyskretnej – simulate_dp.m**

```
function [omega_traj, T_traj] = simulate_dp(p, Uopt)
    omega_traj = zeros(1, p.N + 1);
    T_traj = zeros(1, p.N);
    omega_traj(1) = p.omega_0;
    for k = 1:p.N
        [~, i] = min(abs(p.omega_grid - omega));
        T = Uopt(i, k);
        T_traj(k) = T;
        omega_traj(k+1) = model_discrete(omega_traj(k), T, p);
    end
end
```

g) Skrypt wyświetlający wyniki symulacji w postaci charakterystyk – plots.m

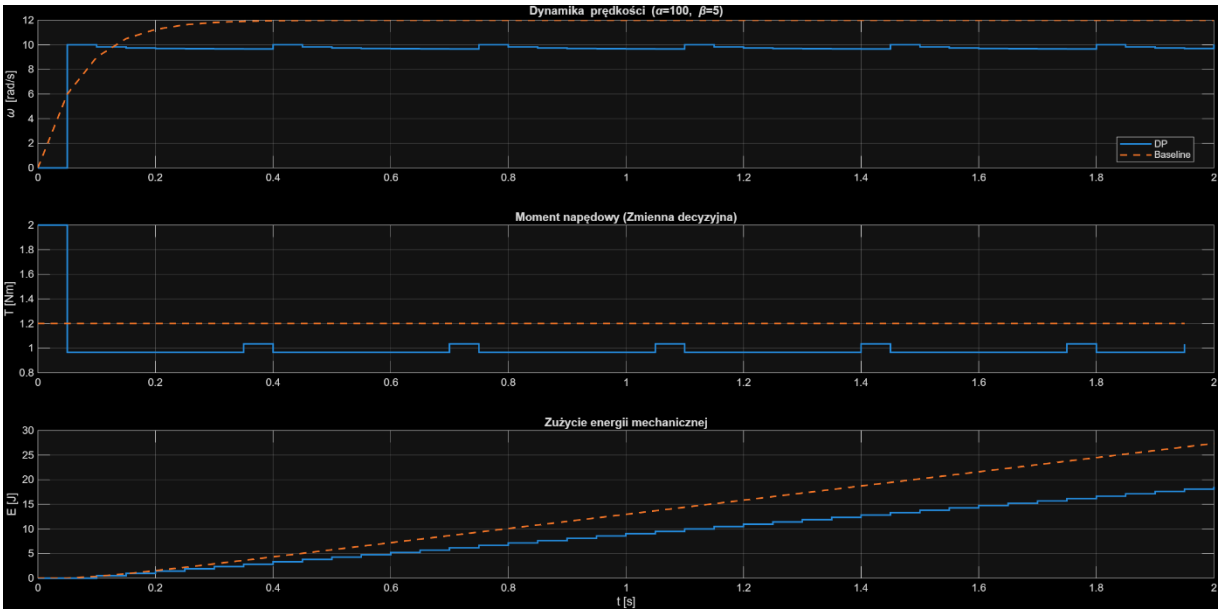
```
function plots(p, omega_dp, T_dp, omega_base, T_base)
    t_state = (0:p.N) * p.dt;
    t_ctrl  = (0:p.N-1) * p.dt;

    figure;
    subplot(3,1,1);
    stairs(t_state, omega_dp, 'LineWidth', 1.5); hold on;
    plot(t_state, omega_base, '--', 'LineWidth', 1.5);
    title(['Dynamika prędkości (\alpha=', num2str(p.alpha_terminal), ', \beta=',
num2str(p.beta), ')']);
    ylabel('\omega [rad/s]');
    legend('DP', 'Baseline');
    grid on;

    subplot(3,1,2);
    stairs(t_ctrl, T_dp, 'LineWidth', 1.5); hold on;
    plot(t_ctrl, T_base, '--', 'LineWidth', 1.5);
    title('Moment napędowy (Zmienna decyzyjna)');
    ylabel('T [Nm]');
    legend('DP', 'Baseline');
    grid on;

    subplot(3,1,3);
    E_dp = cumsum(T_dp .* omega_dp) * p.dt;
    E_base = cumsum(T_base .* omega_base) * p.dt;
    stairs(t_state, E_dp, 'LineWidth', 1.5); hold on;
    plot(t_state, E_base, '--', 'LineWidth', 1.5);
    title('Zużycie energii mechanicznej');
    ylabel('E [J]');
    xlabel('t [s]');
    legend('DP', 'Baseline');
    grid on;
end
```

h) Wyniki symulacji


Dynamika prędkości (α=100, β=5)
ω [rad/s]
DP
Baseline
Moment napędowy (Zmienna decyzyjna)
T [Nm]
Zużycie energii mechanicznej
E [J]
t [s]

4. Projekt – wersja z modelem Simulink

a) Skrypt główny – main.m

```
clear;
clc;

p = params();
Uopt = dp_solver(p);
[w, T, E, t] = run_simulink(p, Uopt, p.T_base_val);
plots(p, w.dp(:)', T.dp(1:end-1)', E.dp(:)', w.base(:)',
T.base(1:end-1)', E.base(:)');
```

b) Skrypt określający parametry projektu – params.m

```
function p = params()
  p.J  = 0.01;
  p.B  = 0.1;
  p.dt = 0.05;
  p.omega_0 = 0;
  p.T_base_val = 1.2;
  p.beta = 5;
  p.Tmax = 2;
  p.omega_max = 12;
  p.omega_target = 10;
  p.N = 40;
  p.omega_grid = linspace(0, p.omega_max, 60);
  p.T_grid     = linspace(0, p.Tmax, 30);
  p.alpha_terminal = 100;
end
```

Opis parametrów:

p.J - moment bezwładności wału napędowego – określa opór układu przy próbie zmiany prędkości

p.B – współczynnik tarcia lepkościowego – reprezentuje straty mechaniczne

p.dt – krok czasu – określa częstotliwość podejmowania przez algorytm decyzji o zmianie momentu napędowego

p,omega_0 – warunek początkowy – określa wartość prędkości na początku symulacji

p.T_base_val – stała moment napędowy – określa stałą wartość momentu napędowego

p.beta – waga kary – określa wagę kary jaką ponosi układ za błąd trakcie ruchu

p.Tmax – saturacja – określa maksymalny dopuszczalny moment napędowy

p.omega_max – zakres prędkości – określa zakres prędkości wykorzystywany przy budowie siatki stanów

p.omega_target – prędkość zadana – określa punkt, który napęd powinien osiągnąć na końcu procesu

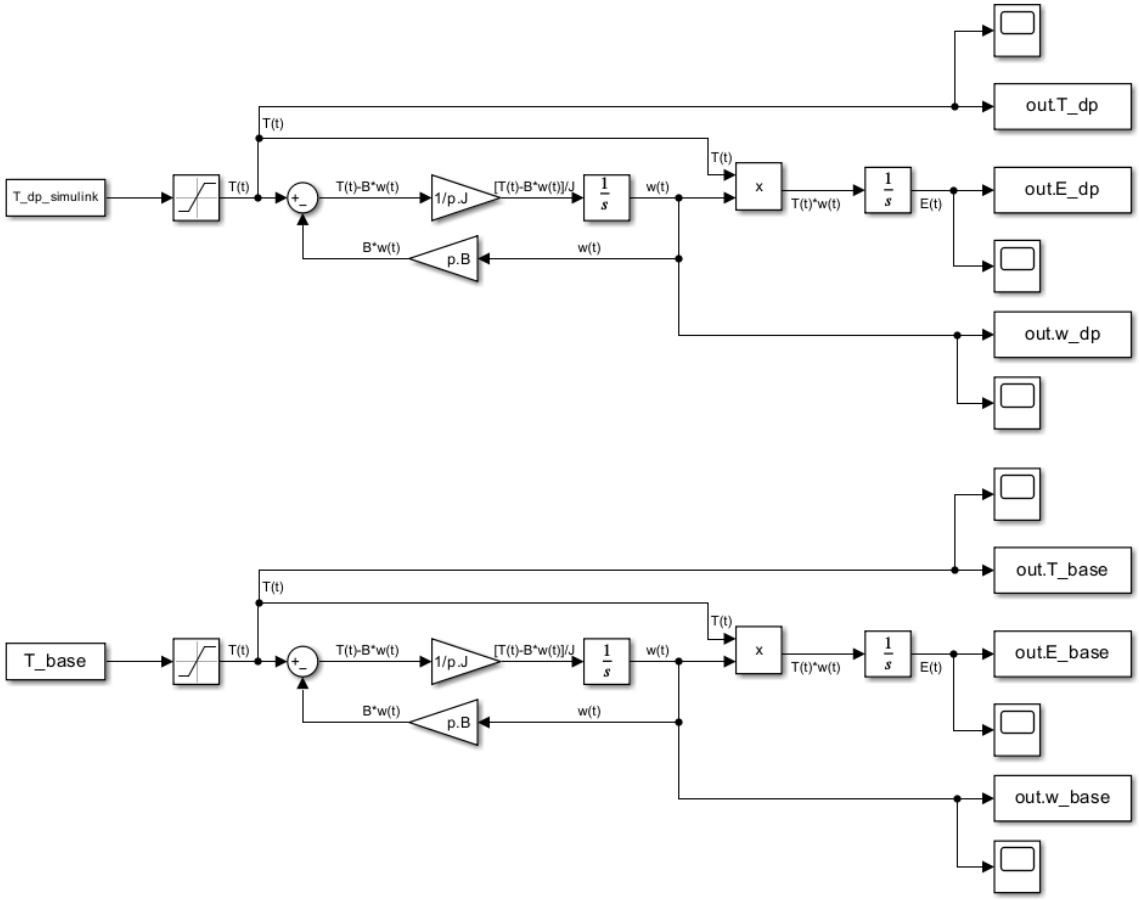
p.N – horyzont czasowy – określa liczbę kroków wykonywanych podczas symulacji. Całkowity czas symulacji to p.N * p.dt

p.omega_grid – siatka prędkości – określa zbiór dopuszczalnych stanów, które może osiągnąć napęd podczas symulacji

p.T_grid – siatka momentów – określa zbiór dopuszczalnych decyzji sterujących, które algorytm może wybrać w trakcie symulacji

p.alpha_terminal – waga kosztu końcowego – decyduje czy ważniejsze jest osiągnięcie idealnej wartości prędkości docelowej (wysoka wartość), czy oszczędność energii (niska wartość)

c) Model Simulink symulujący napęd przed optymalizacją oraz po optymalizacji – system_model.slx



d) Skrypt obsługujący model Simulink – run_simulink.m

```
function [w_sim, T_sim, E_sim, t_sim] = run_simulink(p, Uopt,
T_base_val)
    model_name = 'system_model';

    t_input = (0 : p.N)' * p.dt;

    T_dp_values = zeros(p.N, 1);
    curr_w = p.omega_0;
    for k = 1:p.N
        [~, idx] = min(abs(p.omega_grid - curr_w));
        T = Uopt(idx, k);
        T_dp_values(k) = T;
        curr_w = curr_w + p.dt*(1/p.J)*(T - p.B*curr_w);
    end

    T_dp_data = [t_input, [T_dp_values; T_dp_values(end)]];
    T_base_data = [t_input, ones(p.N + 1, 1) * T_base_val];

    load_system(model_name);
    set_param(model_name, 'SolverType', 'Fixed-step');
    set_param(model_name, 'Solver', 'ode1');
    set_param(model_name, 'FixedStep', num2str(p.dt));
    set_param(model_name, 'StopTime', num2str(p.N * p.dt));

    assignin('base', 'T_dp_simulink', T_dp_data);
    assignin('base', 'T_base', T_base_data);

    sim_out = sim(model_name, 'ReturnWorkspaceOutputs', 'on');

    t_sim = sim_out.tout;
    w_sim.dp   = sim_out.find('w_dp');
    T_sim.dp   = sim_out.find('T_dp');
    E_sim.dp   = sim_out.find('E_dp');
    w_sim.base = sim_out.find('w_base');
    T_sim.base = sim_out.find('T_base');
    E_sim.base = sim_out.find('E_base');
end
```

e) Skrypt rozwiązujący problem optymalizacji dynamicznej – dp_solver.m

```
function [w_sim, T_sim, E_sim, t_sim] = run_simulink(p, Uopt, T_base_val)
    model_name = 'system_model';
    t_input = (0 : p.N)' * p.dt;

    T_dp_values = zeros(p.N, 1);
    curr_w = p.omega_0;
    for k = 1:p.N
        [~, idx] = min(abs(p.omega_grid - curr_w));
        T = Uopt(idx, k);
        T_dp_values(k) = T;
        curr_w = curr_w + p.dt*(1/p.J)*(T - p.B*curr_w);
    end

    T_dp_data = [t_input, [T_dp_values; T_dp_values(end)]];
    T_base_data = [t_input, ones(p.N + 1, 1) * T_base_val];

    load_system(model_name);
    set_param(model_name, 'SolverType', 'Fixed-step');
    set_param(model_name, 'Solver', 'ode1');
    set_param(model_name, 'FixedStep', num2str(p.dt));
    set_param(model_name, 'StopTime', num2str(p.N * p.dt));

    assignin('base', 'T_dp_simulink', T_dp_data);
    assignin('base', 'T_base', T_base_data);

    sim_out = sim(model_name, 'ReturnWorkspaceOutputs', 'on');
    t_sim = sim_out.tout;
    w_sim.dp   = sim_out.find('w_dp');
    T_sim.dp   = sim_out.find('T_dp');
    E_sim.dp   = sim_out.find('E_dp');
    w_sim.base = sim_out.find('w_base');
    T_sim.base = sim_out.find('T_base');
    E_sim.base = sim_out.find('E_base');
end
```

f) Skrypt wyświetlający wyniki symulacji w postaci charakterystyk – plots.m

```
function plots(p, omega_dp, T_dp, E_dp, omega_base, T_base, E_base)
    omega_dp = omega_dp(:)'; omega_base = omega_base(:)';
    T_dp = T_dp(:)'; T_base = T_base(:)';
    E_dp = E_dp(:)'; E_base = E_base(:)';
    t_omega = (0:length(omega_dp)-1) * p.dt;
    t_T     = (0:length(T_dp)-1) * p.dt;
    t_E     = (0:length(E_dp)-1) * p.dt;

    figure('Name', 'Porównanie DP vs Baseline (Simulink)',
'NumberTitle', 'off');

    subplot(3,1,1);
    stairs(t_omega, omega_dp, 'LineWidth', 1.5); hold on;
    plot(t_omega, omega_base, '--', 'LineWidth', 1.5);
    title(['Dynamika (\alpha=', num2str(p.alpha_terminal), ',
\beta=', num2str(p.beta), ')']);
    ylabel('\omega [rad/s]');
    legend('DP', 'Baseline', 'Location', 'best');
    grid on;

    subplot(3,1,2);
    stairs(t_T, T_dp, 'LineWidth', 1.5); hold on;
    plot(t_T, T_base, '--', 'LineWidth', 1.5);
    title('Moment napędowy (Zmienna decyzyjna)');
    ylabel('T [Nm]');
    grid on;

    subplot(3,1,3);
    stairs(t_E, E_dp, 'LineWidth', 1.5); hold on;
    plot(t_E, E_base, '--', 'LineWidth', 1.5);
    title('Zużycie energii mechanicznej');
    ylabel('E [J]');
    xlabel('t [s]');
    grid on;
end
```

g) Wyniki symulacji

